KernAdapt：面向不同目标的组合内核动作学习与选择后风险控制

摘要

可编程内核机制为智能体提供了多种资源控制手段，但如何根据运行状态与用户目标选择组合配置，仍是一个独立于调度器设计的问题。候选方案之间的效果会相互影响，从多个预测中选择最优者还可能放大预测误差。本文提出 KernAdapt，根据系统观测预测组合配置的多项性能指标，并判断是否调整 Linux 默认配置。

系统以 Linux 为基线，通过分组随机实验训练结果预测、配对效用和局部检索模型，再根据动作参数、用户目标及模型之间的差异组合预测。只有结果预测模型负责计算约束余量，排序模型不能修改这一结果。候选生成、排序和拒绝执行共同构成选择策略；风险按随机实验组统计，可靠性判断只能拒绝已选方案。模型输出动作方案，参数、权限、持续时间和异常恢复由独立执行模块管理。

在预先确定方案的 E1 评测中，560 个实验组完成 2,800 次动作执行。完整 ActionPlan 不参与对应训练时，KernAdapt 的平均 regret 为 0.04357，相对因子化结果预测 GBDT 和 pMVX 式近邻检索分别降低 30.95% 和 35.36%；按实验组配对的差值 95% 置信区间分别为 [0.01616,0.02307] 和 [0.02006,0.02786]。

另一组五类真实工作负载包含 600 次决策。Agent 在不知道实际结果时作出选择，147 次调整资源，执行比例为 24.50%，其单侧 90% 下界为 22.23%；实际执行中 77.55% 获得正收益。将保持 Linux 的决策按零增益计入后，平均有界效用增益为 0.01398，按完整轮次分组重采样的 95% 置信区间为 [0.01094,0.01694]。147 次实际执行中，违反任一约束及严重退化风险的精确单侧 90% 上界分别为 3.58% 和 1.55%。模型和阈值在评测期间保持不变，结论适用于 E1 的任务分布及训练样本覆盖的动作因子。

1 引言

AI 智能体的核心能力是围绕状态、目标、工具和环境反馈形成可修正的决策循环，而不只是生成自然语言 [20, 7]。这一能力在操作系统资源控制中尤为重要。相同工作负载可能在 CPU 饱和、锁竞争、长尾请求和后台竞争阶段之间切换；相同内核动作也会随用户强调吞吐、尾延迟或公平性而改变价值。可编程调度、cgroup 和 CPU 亲和性已经提供了足够丰富的控制原语，但它们并不回答何时使用何种机制组合。

近期工作从三个方向缩小了这一差距。学习式路由器根据工作负载类别查询专家调度器映射 [19]；策略记忆通过并行多版本执行积累工作负载到调度策略的检索关系 [14]；LLM 控制面则分析负载并生成、验证 sched_ext 代码 [21]。这些方法表明智能体参与内核控制具有现实价值。与此同时，粗粒度的 scheduler identity 难以表达 placement、slice、preemption、affinity、quota 和 weight 之间的组合，查表或检索也无法在用户目标改变后重新解释同一组多指标结果。本文研究的对象因此不是新的调度器，而是智能体在结构化内核工具上的效果学习与选择问题。

这一问题包含三个相互关联的困难。第一，组合动作具有结构但不是简单相加。例如，短时间片是否有益取决于抢占模式，CPU 放置是否有效又受亲和性和资源份额约束；将每个完整动作当成独立类别会浪费数据，而无约束的全交互模型容易对未观测组合产生乐观外推。第二，真实系统干预昂贵且只提供不完整反馈。顺序执行还受到温度、缓存、DVFS 和工作负载漂移影响，不能把跨时运行直接当成同一时刻的精确潜在结果。第三，即使每个候选分别具有良好的边际预测区间，arg max 也会偏向预测误差最乐观的候选；候选越多，这种选择后偏差越明显。安全判断若忽略候选搜索过程，可能在模型最自信时执行错误动作。

KernAdapt 围绕三个设计思路构造本地 AI 智能体。其一，使用结构化动作因子学习与目标无关的逐指标效果，而不使用动作名称。其二，独立的配对 Critic 从候选–Linux 干预中直接学习有界的与用户目标相关的效用；目标可以影响排序，却不能让排序模型虚构安全的指标裕量。其三，校准完整选择策略：候选生成、排序、arg max 和接受/拒绝被视为同一目标策略，风险统计以随机实验组而非动作或重复目标为单位。

KernAdapt 保留了围绕观测、结构化目标、工具、真实反馈、记忆和重决策的智能体接口。学习策略生成合法动作方案（ActionPlan）或保持 Linux，独立反馈路径把真实结果写入后续决策所需的记录。为隔离学习决策本身，本文留出评测固定模型参数和阈值，不用测试结果在线更新策略。模型没有执行权限：确定性验证、Guard、Canary、Lease 和 Rollback 限制动作的合法性、持续时间和失败影响。该分离使统计风险回答“预测最优动作是否值得执行”，运行时执行模块回答“该动作是否被授权以及如何恢复”。

本文作出以下贡献：

1. 将本地内核资源控制形式化为面向不同目标的组合工具学习问题。Agent 维护系统观测、短期反馈与可查阅的运行记录；动作是上下文相关的结构化 ActionPlan；结果是多指标向量；策略可以显式拒绝并回退 Linux。学习特征均不含动作执行后信息：结果预测模型读取数值状态和结构化动作因子，Critic 另外读取结构化目标及工作负载与配置标识；两个学习模型均不读取 ActionPlan ID 或候选位置。
2. 提出因子化多专家组合策略学习器。不使用动作名称和环境标签的结果预测模型预测供约束检查使用的逐指标效果；根据结构化目标工作的配对 Critic 从随

机候选–Linux 证据中学习动作排序; 局部干预记忆提供检索归纳偏置。组合模型使用结构化动作与目标交互组合这些专家，同时排除完整动作、工作负载和配置身份；约束判定权仍只属于结果路径。

3. 提出面向自适应动作选择的组级多约束风险控制。该方法在随机不完全块反馈下，同时估计非 Linux 动作覆盖率、联合违规风险和执行条件违规风险，并允许在有限阈值族上选择效用最高的可行参数配置；下游可靠性模型只能把已选动作改为拒绝, 不能增加候选、替换动作或获得执行权。
4. 构建由五类真实工作负载、八选一在线动作目录、八组合完整留出基准实验、eBPF 感知和可信执行边界组成的本地 Agent 环境，通过 560 组组合评测和独立的 600 次固定策略决策实验，分析排序、选择性效用、约束风险和高权限动作的影响边界。

2 问题定义

2.1 决策上下文与合法动作

在决策时刻 t，智能体接收上下文 $x_t = (o_t, m_t, e_t)$、结构化目标 $g_t \in \mathcal{G}$ 和机制约束 $\mathcal{C}_t$。其中 o_t 包含记录来源和采集时间的 eBPF、PSI、procfs/cgroup 和应用观测; m_t 记录上一动作、真实结果、Critic 判断和阶段历史; e_t 绑定机器、内核、工作负载与程序版本。确定性适用性函数构造合法集合

$$\mathcal{A}_t = \mathcal{A}(x_t, g_t, \mathcal{C}_t), \qquad a_0 \in \mathcal{A}_t, \tag{1}$$

其中 a_0 表示 Linux 默认策略。非法组合在模型评分前即被排除，而不是依赖模型赋予其较低概率。

定义 1 (动作方案)**.** 动作方案 a 是一个满足约束约束的结构化元组

$$\begin{aligned} a = (&z_{\text{sched}}, z_{\text{queue}}, z_{\text{place}}, z_{\text{preempt}}, z_{\text{idle}}, \\ &p_{\text{slice}}, z_{\text{affinity}}, p_{\text{quota}}, p_{\text{weight}}). \end{aligned} \tag{2}$$

离散字段表示机制开关, 数值字段被约束量化到有界桶。动作方案不包含 *shell* 命令、文件路径或可执行代码。

运行记录中的机器、内核、工作负载、配置和程序版本共同服务于来源审计、适用性判断和 Guard 授权，但两个学习模型的输入并不相同。结果预测模型只接收未施加动作的数值观测、缺失掩码和结构化动作因子，不读取环境标签。Critic 另外接收目标 g_t、workload-family 指示量、configuration-ordinal 指示量及其与动作的交互。两个学习模型均排除动作名称、ActionPlan ID 和候选位置；机器、内核与程序版本也不进入任一学习模型。该披露使来源绑定与真实的学习条件保持清晰分离。

2.2 多指标效果与目标效用

动作 a 在上下文 x 下产生向量结果

$$\mathbf{Y}(a) = [q_{.99}, T, B, S, F, C_{\text{ctl}}], \tag{3}$$

依次表示 P99 延迟、主任务吞吐、后台进度、资源份额、公平性和控制成本。本文学习相对 Linux 的条件效果。对正值性能指标，定义 Linux 相对条件对数效果

$$\begin{aligned} \boldsymbol{\Delta}_m(x,a) &= \mathbb{E}[\log Y_m(a) - \log Y_m(a_0) \mid x], \\ \boldsymbol{\Delta}(x,a_0) &= \mathbf{0}. \end{aligned} \tag{4}$$

数据来自相同的执行前状态附近的顺序随机干预，因此本文称其为配对干预效果，而不把它解释为同一时刻全部动作的个体反事实真值。

结构化目标 g 给出指标权重和 SLO 阈值。令 $D = (D_T, D_L, D_B)$，训练目标、候选排序和主评测统一采用有界对称效用

$$\begin{aligned} u_g(D) = w_T \tanh D_T + w_L \tanh D_L + w_B \tanh D_B \\ - w_F \tanh |D_T - D_B|. \end{aligned} \tag{5}$$

其中延迟权重为负，不可观测 P99 由掩码移出效用。$e^{D_m} - 1$ 只用于把原始结果展示为相对百分比，不作为训练或策略评测目标；记 $U_g(x,a) = u_g(\boldsymbol{\Delta}(x,a))$。由于效果模型不读取目标，同一组 $\widehat{\boldsymbol{\Delta}}(x,a)$ 可在目标权重或 SLO 松紧改变后重新组合, 无需重新拟合数值结果模型。

2.3 约束余量、选择性策略与风险

每个统计约束被转化为“正值满足、负值违反”的约束余量

$$M_j(x,a,g) = h_j(\mathbf{Y}(a), g), \qquad j = 1, \ldots, J. \tag{6}$$

例如，P99 上界 $L_{\max}$ 对应 $M_{\text{P99}} = \log L_{\max} - \log q_{.99}$，吞吐下界 $T_{\min}$ 对应 $M_T = \log T - \log T_{\min}$。参数合法性、目标身份和回退可用性属于硬前置条件，不与性能约束混成单一风险标签。

策略输出一个合法动作或拒绝:

$$\pi(x,g) \in \mathcal{A}(x,g) \cup \{\perp\}. \tag{7}$$

$\perp$ 在部署时等价于 a_0,但动作覆盖率 $C(\pi) = \mathbb{P}(\pi(x,g) \neq a_0)$ 单独报告。相对 Linux 的策略效用和观测动作后悔分别为

$$J(\pi) = \mathbb{E}[U_g(x, \pi(x,g)) - U_g(x, a_0)], \tag{8}$$

$$\text{Regret}_r(\pi) = \max_{a \in S_r} U_g(x_r, a) - U_g(x_r, \pi(x_r, g)), \tag{9}$$

其中 S_r 是评测实验组 r 中实际执行的动作子集，故式 (9)是已测动作中的事后最优方案 regret，而非无限参数空间的全局最优差距。

令 $A = \mathbb{I}\{\pi(x,g) \neq a_0\}$，$V_j = \mathbb{I}\{M_j < 0\}$。本文区分

$$R_j^{\text{joint}} = \mathbb{P}(A = 1, V_j = 1), \tag{10}$$

$$R_j^{\text{cond}} = \mathbb{P}(V_j = 1 \mid A = 1). \tag{11}$$

前者衡量所有决策中的不安全行动，后者衡量真正采取动作后的质量。两者与 $C(\pi)$ 共同报告，以避免将低行动率造成的小联合风险误解为低条件风险。

3 KernAdapt 智能体

3.1 决策循环

KernAdapt Agent 表示为

$$\mathcal{K} = (E_\phi, F_\theta, Q_\psi, H_\omega, R_\rho, P, \Gamma_\lambda, C_{\text{fb}}, M). \tag{12}$$

观测编码器 E_ϕ 将执行动作前的当前观测编码为 $s(x)$; 效果模型 F_θ 预测每个合法动作的多指标均值、尺度和支持度; 配对效用 Critic Q_ψ 与因子化组合模型 H_ω 根据用户目标对方案排序; 可靠性判断 R_ρ 只能拒绝而不能

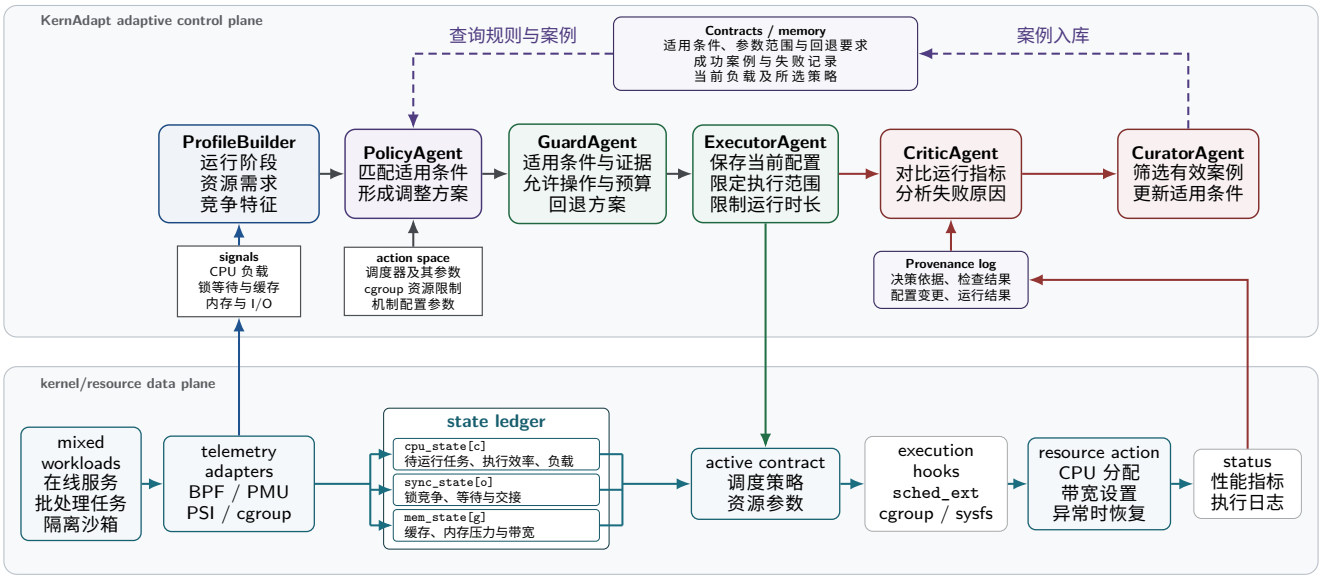


图 1: KernAdapt 运行时架构。用户态 AI 智能体完成观测编码、动作效果预测、根据用户目标制定方案、选择与记忆更新；可信工具边界完成类型验证、权限检查和事务执行；eBPF、sched_ext 与 cgroup 构成可观测可行动的任务环境。模型只产生 ActionPlan 提议，不持有内核执行能力。

替换其提议。Planner P 组合约束余量并搜索合法目录；选择器 Γ_λ 输出 ActionPlan 或 ABSTAIN；反馈 Critic C_{fb} 将真实反馈分解为效用、约束结果、工具状态和证据质量；记忆 M 保存最近阶段及其动作结果。一次交互形成

$$\tau_t = (x_t, g_t, \mathcal{A}_t, a_t, \widehat{\mathbf{Y}}_t, \mathbf{Y}_t, c_t, x_{t+1}), \tag{13}$$

从而将一次性的内核调参转化为可重放的 Agent trajectory。

反馈与记忆组件定义了 AI 智能体的重决策接口。第 6节的评测有意使用固定的逐次决策策略：outcome 沿该接口记录，但在留出运行期间不更新模型权重或选择阈值。因此，本文报告的收益评估从观测、目标、方案选择到实际执行的全过程，不单独归因于在线适应或记忆更新。

3.2 观察、规划与记忆

原始内核事件流首先由观测适配器聚合为固定维度统计量，包括 runnable pressure、wakeup 与 handoff 局部性、CPU 忙闲分布、PSI、cgroup 份额以及应用层吞吐和尾延迟。每个特征均绑定时间窗口、缺失掩码和来源。Planner 仅在约束验证后的候选上运行；观测缺失既增加模型不确定性，也可能直接移除依赖该观测的动作。短期记忆提供阶段变化、上一动作、Canary 结果和重复失败信息，但固定模型评测期间不在线更新模型权重。

自然语言目标由低频语义模块解析为结构化目标。该模块可以是规则解析器、本地小型编码器或经过离线蒸馏的语言模型；高频数值控制路径只读取验证后的字段。由此，网络不可用不会中断 Agent 决策，语言模型也不能绕过动作 schema 或直接写入内核参数。

3.3 可信工具边界

图 1中的可信边界将学习决策与执行授权分离。Verifier 检查观测、目标、约束、动作列表和模型文件身份；Guard 重放参数范围、组合约束、目标进程树和资源预算；Executor 以事务方式保存执行前状态并应用调度与 cgroup 参数；Canary 监视硬停止条件；Lease 和 watchdog 限制动作持续时间，Rollback 恢复 Linux 调度状态、CPU 亲和性与 cgroup 快照。这一边界处理能力安全与故障暴露，不修改效果模型的排序，也不被计作学习收益。

4 组合动作效果学习

4.1 分组随机干预

数据收集以随机实验组为单位。给定相同的执行前状态 x_r，Linux 基线必定进入块，其他合法动作依据预先给定的包含概率 $p_r(a) > 0$ 抽样；动作顺序随机化，动作之间执行清理、残留状态清理和固定时长的冷却。记录包含概率、执行顺序、执行前状态、温度/DVFS 状态、指标缺失掩码和最终调度状态。完整块才进入学习，使候选与基线差值

$$D_{r,a,m} = \log \frac{Y_{r,m}(a)}{Y_{r,m}(a_0)} \tag{14}$$

共享尽可能接近的系统环境。随机化提供已知 propensity，Linux 基线则消除一部分块级公共漂移。

4.2 结构化因子与对训练样本覆盖的配置组合进行预测

令 $s(x) \in \mathbb{R}^{d_s}$ 编码未施加动作的的进程局部信号及缺失掩码，$z(a)$ 拼接式 (2)中 scheduler、placement、queue、slice、preemption、affinity、quota 和 weight 字段的 one-hot 编码。动作名称、模板标签和候选位置均不进入两个学习模型。训练索引分别统计每个因子水平，以及 queue–placement、slice–preemption、placement–affinity 和 quota–weight 四类机制相关因子对。只有当全部必要因子水平和登记因子对在多个随机块及配置中获得支持时，候选才进入学习评分；完整配置组合本身可以在训练中出现零次。

状态支持同样只使用干预前信息。我们在训练组上标准化 $s(x)$，计算每个训练组的 leave-one-group-out 最近邻距离。推理查询超过预先确定的第 99 百分位阈值时，在模型评分前直接拒绝并保持 Linux。该支持包络不

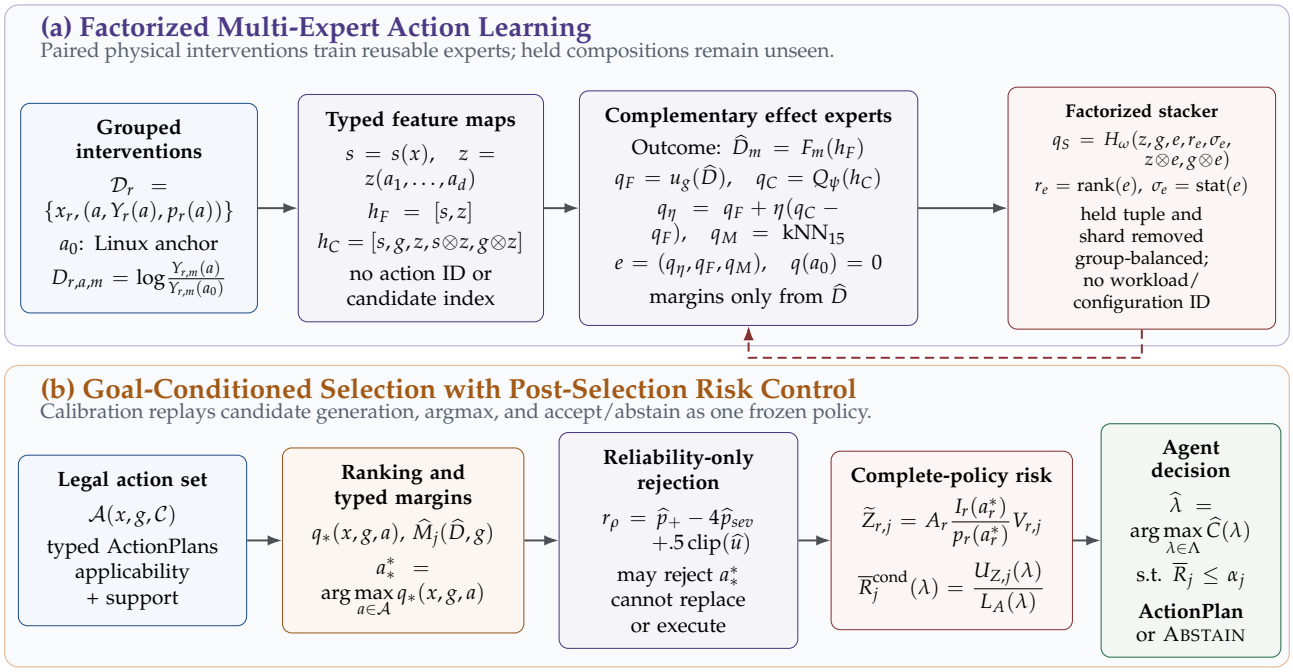


图 2: KernAdapt 学习与选择算法。以 Linux 为基线的随机不完全块提供候选–基线配对效果；结构化因子进入结果、配对效用和检索专家，因子化组合模型组合其互补分数以预测留出完整动作。由结果预测得到的约束余量、完整策略风险校准及只能拒绝的可靠性判断共同决定执行已排序 ActionPlan 还是保持 Linux。

是通用 OOD 检测器，而是声明学习策略允许外推的总体。

4.3 向量结果预测模型

对 $m \in \{T, L_{.99}, B\}$,结果预测模型是 CPU histogram gradient-boosted regressor：

$$\widehat{D}_m(x, a) = F_m([s(x), z(a)]), \qquad \widehat{D}_m(x, a_0) = 0. \quad (15)$$

非线性树可以学习结构化因子列之间的交互，但不能通过 ID 列记忆被留出的 ActionPlan。给定已知包含概率 $p_r(a)$ 和指标掩码 $\omega_{r,a,m}$，训练最小化

$$\mathcal{L}_F = \sum_{r,a,m} \omega_{r,a,m} \min\{p_r(a)^{-1}, 4\} \cdot \left(D_{r,a,m} - F_m([s_r, z_a])\right)^2. \quad (16)$$

按实验组进行五折交叉拟合得到 out-of-fold 残差；第二个回归器预测残差绝对值的对数。指数变换、正值下限和互斥的尺度校准共同产生有限且为正的 $\widehat{\sigma}_m(x, a)$。这是 mean-plus-scale 构造，并不假设物理系统噪声严格服从高斯分布。

结构化目标将 $\widehat{D}_m$ 确定性地组合为式 (6)的带正负号的约束余量。因此排序模型不能将不安全指标宣称为安全；只有结果路径能够向选择器和 Guard 提供裕量。

4.4 面向不同目标的配对 Critic

结果组合与动作排序不必具有相同统计目标。直接回归原始指标可能把容量消耗在与当前目标无关的数值精度上，而单一标量动作价值又会丢弃约束检查需要复用的逐指标证据。KernAdapt 同时保留二者。对目标向量 g，定义有界配对目标

$$u_g(D) = w_T \tanh D_T + w_L \tanh D_L + w_B \tanh D_B - w_F \tanh |D_T - D_B|, \quad (17)$$

P99 不可观测时屏蔽延迟项。这样，互为倒数的加速与减速具有对称幅度，单次极端运行也不能主导排序损失。

Critic 特征映射为

$$h_C(x, g, a) = [s(x), g, z(a), o_{.99}, s(x) \otimes z(a), g \otimes z(a), e_w, e_c, e_w \otimes z(a), e_c \otimes z(a)], \quad (18)$$

其中 e_w 和 e_c 分别是工作负载 family 与配置 ordinal 指示量。配对 Critic Q_ψ 最小化

$$\mathcal{L}_C = \sum_{r,g,a \neq a_0} \min\{p_r(a)^{-1}, 4\} \cdot \left(u_g(D_{r,a}) - Q_\psi(h_C(x_r, g, a))\right)^2, \quad (19)$$

并硬锚定 $Q_\psi(x, g, a_0) = 0$。其尺度由实验组交叉拟合残差和互斥的 90% 归一化残差分位数得到。Critic 只暴露 $(\widehat{q}_g, \widehat{s}_g)$，没有逐指标输出、约束判定权或执行 API。动作名称、ActionPlan ID、候选位置以及机器/内核/程序版本均不进入 Critic 特征。

4.5 结果锚定的动作排序

两个学习模型提供互补估计。令 $q_F(x, g, a) = u_g(\widehat{D}(x, a))$ 表示由多指标预测确定性组合得到的效用，$q_C(x, g, a) = Q_\psi(h_C(x, g, a))$ 表示直接配对效用。KernAdapt 采用收缩分数

$$q_\eta(x, g, a) = q_F + \eta(q_C - q_F), \quad \eta = 0.9, \quad (20)$$

其中 η 在打开留出结果前由 development 实验组固定。$\eta = 0$ 退化为 factor-only 结果组合，$\eta = 1$ 退化为直接配对 Critic；部署值保留 90% 的根据目标计算的 Critic 修正，同时向可解释的结果路径收缩。尺度使用相同凸权重组合，随后在互斥实验组上校准。该分数只影响排序和效用下界；全部结构化约束余量仍只由 F_m 产生。

命题 1 (动作名称不变性)**.** 固定 (x, g) 时，在不改变任何结构化字段的前提下重命名 *ActionPlan*，不会改变式 (15)与式 (18)。因此两个学习模型都不能用名义动作身份解决完整组合留出任务。

该命题直接来自 $z(a)$ 只依赖结构化字段。它并不声称任意未见组合都可预测；因子/因子对支持及状态包络共同决定何时允许外推。

4.6 因子化专家 stacking

逐指标结果组合、直接配对回归和局部检索在不同状态下具有不同误差。KernAdapt 不在全局固定选择一个专家，而令 $e(x, g, a) = (q_\eta, q_F, q_M)$ 分别表示结果锚定 Critic、因子化结果预测和 15 近邻历史记录分数，并预测

$$q_S(x, g, a) = H_\omega([z(a), g, e, \mathrm{rank}(e), \mathrm{stat}(e), z(a) \otimes e, g \otimes e]). \tag{21}$$

$\mathrm{stat}(e)$ 汇总同一候选集合内各专家的均值、离散度和相对秩。H_ω 使用按实验组平衡权重的 ExtraTrees 回归，不接收完整动作 ID、工作负载或配置身份。每个组合 fold 在拟合前删除目标 ActionPlan 的全部标签及其 collection shard；模型只能通过其他合法组合共享因子证据。组合模型有排序权，但没有约束判定权或执行权。

保守实机决策实例复用相同结果与配对专家，并用局部干预历史记录正则化锚定分数：

$$q_R = (1 - \lambda_w) q_\eta + \lambda_w q_M. \tag{22}$$

开发阶段确定的负载分组权重 λ_w 只改变两个专家的可信比例，不把工作负载直接映射到某个动作。完整组合实验评测 q_S，实际运行实验评测 q_R；两者的模型与参数均在评测前固定。组合模型的 regret 改善与实际运行收益分别统计。

4.7 只能拒绝的可靠性判断

q_R 选出方案后，R_ρ 只读取未施加动作的状态、结构化目标与约束、动作因子和决策时已有的预测诊断，交叉拟合正收益、任一约束、严重事件概率及期望效用。固定的 admission score 为

$$r_\rho = \widehat{p}_+ - 4\widehat{p}_{\text{severe}} + 0.5\,\mathrm{clip}(\widehat{u}, -0.2, 0.2). \tag{23}$$

阈值只根据开发集预测选择，并在生成实际运行评测安排前一次性重新拟合。该门只能将已有非 Linux 提议变为 Abstain；它不能增加候选、替换动作、修改约束余量或执行工具。

4.8 搜索与选择性拒绝

对有界合法目录，Planner 选择

$$a_*^*(x, g) = \arg\max_{a \in \mathcal{A}(x,g)} q_*(x, g, a). \tag{24}$$

q_* 表示针对相应 estimand 预先注册的排序器。仅当该方案校准后的效用下界为正，全部由结果预测模型计算的约束余量与风险预算均通过，且可靠性判断接受时，才执行被选方案；否则输出 Abstain 并保持 Linux。更大动作空间可以使用确定性的候选生成器替代枚举，但校准必须完整重放部署时使用的生成器和优化器。

5 选择后多约束风险控制

5.1 校准对象：完整选择策略

逐候选的边际区间并不自动转化为被选动作的风险保证。若 K 个候选的预测误差近似独立，选择最大预测效用相当于选择最乐观误差；即使每个候选以 $1 - \alpha$ 概率覆盖，所有候选同时覆盖的概率也可能降至 $(1-\alpha)^K$。Bonferroni 修正能够恢复同时覆盖，但在候选较多时会产生很宽的区间和过度拒绝。

KernAdapt 在风险校准前固定状态编码器、效果模型、结构化目标 composer、候选生成器和 arg max。给定有限阈值族 Λ，策略写为

$$\pi_\lambda(x, g) = \begin{cases} a_*^*(x, g), & B_\lambda(x, g) = 1, \\ a_0, & B_\lambda(x, g) = 0, \end{cases} \tag{25}$$

其中 B_λ 只读取预测效用、约束余量、支持度和预处理状态，不读取当前真实结果。

5.2 随机不完全块中的组级风险估计

对实验组 r 和约束 j，定义策略违规损失

$$Z_{r,j}(\lambda) = A_r(\lambda) \mathbb{I}\{M_j(Y_r(a_r^*), g_r) < 0\}. \tag{26}$$

不完全块未必执行 a_r^*。令 $I_r(a_r^*)$ 表示该动作是否进入随机块，$p_r(a_r^*)$ 为其已知包含概率，则 Horvitz–Thompson 组级 pseudo-loss 为

$$\widetilde{Z}_{r,j}(\lambda) = A_r(\lambda) \frac{I_r(a_r^*)}{p_r(a_r^*)} V_{r,j}(a_r^*). \tag{27}$$

在随机分配和一致性条件下，式 (27)对 R_j^{joint} 无偏。多个动作和多个约束仍先在组内聚合，每个实验组只贡献一个独立统计单位。由于 $p_r(a) \geq p_{\min} > 0$，该组级 pseudo-loss 有界于 $[0, B]$，其中 $B = 1/p_{\min}$。

设风险校准集中有 n 个独立实验组，

$$\widehat{\mu}_{Z,j}(\lambda) = \frac{1}{n} \sum_r \widetilde{Z}_{r,j}(\lambda), \tag{28}$$

$$\widehat{\mu}_A(\lambda) = \frac{1}{n} \sum_r A_r(\lambda). \tag{29}$$

令

$$\eta = \log \frac{2|\Lambda|(J+1)}{\delta}, \tag{30}$$

并构造

$$U_{Z,j}(\lambda) = \widehat{\mu}_{Z,j}(\lambda) + B\sqrt{\frac{\eta}{2n}}, \tag{31}$$

$$L_A(\lambda) = \widehat{\mu}_A(\lambda) - \sqrt{\frac{\eta}{2n}}. \tag{32}$$

于是条件违规风险上界为

$$\overline{R}_j^{\text{cond}}(\lambda) = \frac{U_{Z,j}(\lambda)}{\max\{L_A(\lambda), \epsilon\}}. \tag{33}$$

若 L_A 低于预先指定的最小覆盖率，条件风险声明无效，策略保持 Linux。

定理 1 (有限阈值族上的选择后风险控制)**.** 假设风险校准实验组独立同分布，动作包含概率正确且 $p_r(a) \geq p_{\min} > 0$，策略族 $\{\pi_\lambda : \lambda \in \Lambda\}$ 在读取校准结果前固定。则以至少 $1-\delta$ 的概率，式 (31)和式 (32)对全部 $\lambda \in \Lambda$ 及约束 $j = 1, \ldots, J$ 同时成立。因此，任何根据这些同时界数据自适应选择且满足 $\overline{R}_j^{\text{cond}}(\widehat{\lambda}) \leq \alpha_j$ 的阈值，都满足 $R_j^{\text{cond}}(\pi_{\widehat{\lambda}}) \leq \alpha_j$；同时有 $R_j^{\text{joint}}(\pi_{\widehat{\lambda}}) \leq U_{Z,j}(\widehat{\lambda})$。

证明由有界随机变量的 Hoeffding 不等式和对 $|\Lambda|(J+1)$ 个事件的并集界得到。由于上界对整个阈值族同时成立，选择覆盖率最高的可行阈值不会引入第二次未校准选择。更紧的经验 Bernstein 界或 e-process 可以替换式 (31)，但不改变校准对象必须覆盖完整选择策略这一原则。

5.3 多约束决策规则

KernAdapt 同时控制逐项约束风险和任一约束违规

$$V_{\text{any}} = \mathbb{I}\left\{\min_j M_j < 0\right\}. \tag{34}$$

从 Λ 中选择

$$\widehat{\lambda} = \arg\max_{\lambda \in \Lambda} \widehat{\mu}_A(\lambda) \quad \text{s.t.} \quad \overline{R}_j^{\text{cond}}(\lambda) \leq \alpha_j,\ \forall j. \tag{35}$$

这一规则与逐指标预测区间承担不同职责：结果区间解释某个指标可能落在哪里，式 (35)直接约束自适应搜索后实际执行动作的约束风险。部署选择器不叠加多个语义重复的风险分类器和启发式阈值。

6 实验方法

6.1 任务、动作与目标

实验主机提供 32 个 AMD EPYC 9654 vCPU 和单 NUMA 节点，运行 Linux 7.0.0-29。工作负载包括 schbench、futex mutex/ring、MariaDB Sysbench 只读 OLTP、RocksDB 只读 heavy-tail Get/Scan 以及通过 llama.cpp 运行的 TinyLlama 1.1B CPU 推理。五类任务分别覆盖调度尾延迟、锁同步、状态服务、存储长尾和本地模型服务。每类任务使用多档并发压力，并配有 best-effort 后台负载。

实际运行的动作目录登记 Linux 默认策略、公开的 scx_simple 参考实现以及六个 scx_adapt 资源模板，共八个选择；每个上下文的合法子集由处置前适用性检查重新确定。六个模板分别强调低延迟、吞吐、隔离、公平性、后台进度和效率，并共同暴露 queue、placement、slice、wakeup preemption、idle preference、affinity、CPU quota 和 weight 字段。完整组合留出基准实验另行定义八个 scx_adapt ActionPlan：它们使用相同结构化因子，但每个完整配置组合分别作为一次留出对象，Linux 始终作为配对锚点执行。实验不将其他研究项目的专用调度机制作为 KernAdapt 动作。五种需要 P99 的结构化目标分别强调尾延迟、吞吐、平衡效用、主任务 SLO 与后台进度，以及公平性预算；第六种吞吐–后台目标在 futex 协议中 mask 该任务未定义的 P99 指标。

6.2 数据划分与统计单位

每个物理块执行 Linux 及从合法列表中随机抽取的候选子集。模型拟合、结果尺度校准、风险阈值选择、风险估计与策略评测使用互不重叠的实验组。方法确定后，完整配置组合留出实验在五类工作负载上建立 560 个新实验组。每组执行 Linux 基线，并从八种组合中随机分配四个 ActionPlan，共完成 2,800 次动作执行；每个完整组合出现在 280 个实验组中。拟合模型前，删除目标组合的全部标签及对应采集分片，但组成该动作的参数取值仍由其他合法组合提供训练样本。根据训练数据确定的状态范围，八种动作的样本覆盖率分别为 87.50% 至 99.64%。五个固定随机种子的结果先在实验组内求均值，再进行跨组统计。

实机决策评测在策略和阈值确定后进行 30 轮，每轮包含 20 个工作负载与配置场景，共 600 次决策，五类工作负载各占 120 次。每次决策都在读取实际结果之前完成观测、评分和选择，再以随机顺序运行所选动作与 Linux 基线。保持 Linux 的决策按零相对效用计入式 (8)。效用的置信区间以包含 20 个场景的完整轮次为单位，进行 10,000 次自助重采样；执行比例与执行后的风险以候选方案和 Linux 的配对实验组为单位，使用精确单侧二项界计算。每次运行结束后检查 sched_ext 是否恢复为 disabled，并清理 cgroup、工作负载与控制器进程。

完整配置组合留出实验在同一台 E1 主机上进行，使用五类已观测工作负载和四种配置。该实验评价已有参数取值组成新配置时的预测能力，不代表对新工作负载、未知配置或其他主机的泛化能力。

6.3 指标

策略质量报告动作覆盖率、所选动作正向收益率、全体决策平均相对效用及组级置信区间。约束质量报告逐项与任一约束违规、严重调度退化、联合风险和执行条件风险。严重事件的零观测结果使用精确 Clopper–Pearson 单侧上界，不将 $0/n$ 写成风险为零。结果模型报告逐指标误差、P99 区间覆盖率，组合学习报告完整组合留出时相对已测最优动作的 regret。所有学习模型使用固定随机种子集合 $\{17, 23, 31, 47, 59\}$，先对同一实验组聚合种子，再进行跨组统计。候选数压力实验明确标记为半合成方法诊断，不与真实系统效用混合。

6.4 比较方法

系统参照包括 Linux、每个固定 ActionPlan、训练集最优固定动作和已测动作中的事后最优方案。三个直接组合基线分别排除不同解释：完整动作编号 GBDT 评估原子动作记忆；pMVX 式 15 近邻历史记录评估状态/目标检索；强因子化结果预测 GBDT 使用相同结构化动作字段预测逐指标结果，但不使用专家 stacking；另外报告实际执行策略使用的 retrieval-regularized paired Critic。选择后压力实验比较逐候选边际校准、Bonferroni 同时区间、CRC 式联合损失控制和本文完整策略校准；同时加入 ASA 式分类查询、同反馈预算的保守 factor ensemble。CPC 需要安全参考策略对所有候选具有非零支持，而确定性 Linux 对非 Linux 动作的支持为零，因此其重要性比未定义，本文不编造数值结果。所有可比较方法接收相同状态、合法候选、缺失掩码、propensity、目标、实验组与反馈预算。

7 实验结果

7.1 E1 任务分布内的有支持完整配置组合留出

表 2在 E1 任务分布内的完整 ActionPlan 留出条件下评测因子化专家组合模型。组合模型后端、支持规则、fold、seed 和分析固定后，才生成 560 个评测实验组并执行 2,800 次动作执行。八个完整计划各在 280 个组中评

表 1: 实机决策策略使用的结构化目标。T, L, B, F 分别表示吞吐、P99 延迟、后台进度和公平性权重；ratio 列是相对 Linux 的硬约束阈值。登记的 control-cost 字段由运行时确定性检查，当前结果效用中取零。

目标	w_T	w_L	w_B	w_F	min T	max L	min B	最大份额差
尾延迟优先	.25	−.65	.10	0	.80	1.10	.70	–
吞吐优先	.70	−.20	.10	0	.90	1.50	.70	–
平衡	.35	−.30	.20	.15	.85	1.25	.75	×2.0
SLO 与后台	.20	−.40	.40	0	.75	1.15	.90	×2.0
公平性预算	.25	−.20	0	.45	.80	1.25	.85	×1.5
吞吐与后台（无 P99）	.50	0	.30	.20	.80	–	.75	×2.0

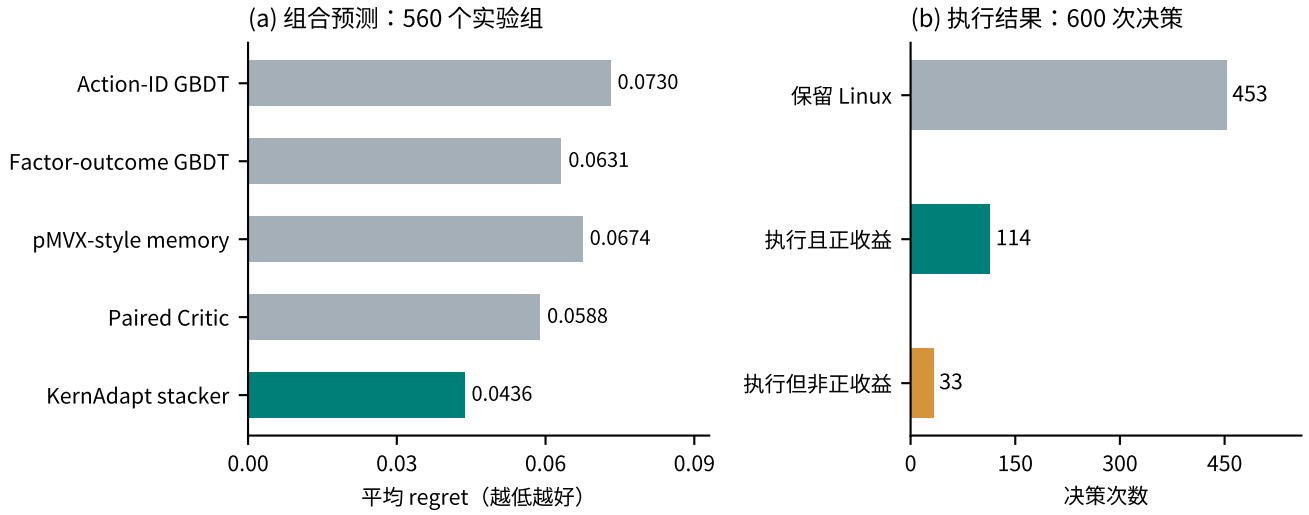


图 3: 固定 E1 评测的两类结果。(a) 完整动作留出实验中的平均 regret，越低越好；(b) 600 次决策中保留 Linux、执行且取得正收益、执行但未取得正收益的次数。两类实验分别使用固定的组合预测模型与执行策略。

测，同时从相应 fold 的全部训练标签和 collection shard 中移除；逐动作状态支持覆盖率介于 87.50% 和 99.64% 之间，支持域外组精确选择 Linux，而不计为成功外推。

KernAdapt 取得 0.04357 的 mean oracle regret。相对强因子化结果预测 GBDT（0.06310），regret 降低 30.95%，实验组配对绝对 reduction 的 95% 置信区间为 [0.01616,0.02307]；相对 pMVX 式 kNN memory（0.06740），regret 降低 35.36%，区间为 [0.02006,0.02786]。相对 action-ID GBDT 和 retrieval-regularized paired Critic 的 regret 还分别降低 40.33% 和 25.92%，两项配对区间也均为正。因此该结果不能由原子动作记忆、仅预测因子化结果或最近邻检索单独解释。组合模型不读取工作负载或配置身份，但这些因子的水平仍来自 E1 任务分布，所以本实验支持完整动作组合留出，而不是跨机器泛化主张。

7.2 与最近非语言模型策略的同预算比较

在另一份 301 个随机化实验组的五 seed 评测中，所有方法只使用相同的日志反馈预算，且完整动作均从其训练 fold 中移除。表 3显示，KernAdapt 相对最强 factor-only GBDT、pMVX 式历史记录和保守三成员 factor ensemble 分别将 regret 降低 15.29%、25.28% 和 26.08%，三项实验组配对区间均为正。ASA 式分类查询由于只预测动作身份而不预测多指标效果，regret 更高。顺序 constrained BO 需要额外的自适应真实系统查询，无法在同一固定反馈预算下公平比较，因此保守 ensemble 作为最接近的同预算优化器。表 2仍是主要的独立完整配置组合结果；因子化组合模型已在该前瞻结果中与因子化结果预测 GBDT 及 pMVX 式检索形成统计分离。

表 2: 完整 ActionPlan 留出上的 mean oracle regret；越低越好。预先确定方案的的一次性评测包含 560 个新实验组、2,800 次动作执行、八个 held ActionPlan 和五个固定 seed。末列给出 KernAdapt 相对各比较方法的实验组配对绝对 regret reduction 之 95% 置信区间。

方法	Mean regret ↓	差值置信区间
KernAdapt 因子化组合模型	**0.04357**	—
检索正则化配对 Critic	0.05881	[0.01228,0.01856]
强因子化结果预测 GBDT	0.06310	[0.01616,0.02307]
pMVX 式 kNN 历史记录	0.06740	[0.02006,0.02786]
Action-ID outcome GBDT	0.07301	[0.02527,0.03381]

结构化目标并非动作类别标签。对留出的连续权重，KernAdapt 在插值目标上的 regret 为 0.02075，而结构化结果组合器为 0.02639，配对 reduction 区间为 [0.00003,0.01139]；在外推权重上分别为 0.01908 和 0.02760，区间为 [0.00280,0.01465]。完全新目标 family 留出尚未形成优势，因此本文的目标主张限定为已表示指标语义内的连续重加权，而非任意新目标理解。四类非法、冲突或不可观测目标均 fail closed；合法目标切换

表 3: 301 个随机化实验组上的同反馈预算比较。五个 seed 先在组内平均，末列为比较方法 regret 减去 KernAdapt regret 的实验组配对 95% 置信区间。

方法	Mean regret ↓	配对差值区间
KernAdapt 锚定 Critic	**0.03137**	—
强 factor-only GBDT	0.03703	[0.00061,0.01085]
pMVX 式 kNN 历史记录	0.04198	[0.00439,0.01707]
保守 factor ensemble	0.04244	[0.00547,0.01701]
ASA 式路由器	0.05260	[0.01417,0.02849]

表 4: 候选数压力实验。违规率均以实际采取动作的样本为分母；预算为 5%。

K	逐候选违规率	完整策略违规率	完整策略 coverage
2	4.84%	2.15%	27.55%
4	5.33%	2.33%	40.96%
8	5.51%	2.57%	55.33%
16	5.61%	2.87%	67.77%
32	5.77%	3.07%	78.13%

表 5: 名义 $\alpha = 0.05$ 下的实验组风险控制比较。风险是在执行动作条件下的任一约束违规；UCB 为精确单侧 90% 上界，效用将拒绝按 Linux 零计入。

方法	Coverage	经验风险	风险 UCB	效用
无外层校准	32.00%	3/128=2.34%	5.14%	0.03934
逐候选边际	3.75%	0/15	14.23%	0.00265
Bonferroni 候选	0.00%	—	—	0.00000
CRC 式联合损失	32.00%	3/128=2.34%	5.14%	0.03934
选择后校准（本文）	**25.00%**	1/100=1.00%	**3.83%**	**0.01286**

后，4,410 项比较中 91.61% 在新目标下效用不下降。

7.3 自适应搜索后的风险控制

我们用保持真实物理效果结构的半合成误差模型，将合法候选数从 2 扩展到 32。表 4显示，逐候选边际校准在 $K \geq 4$ 后超过 5% 条件违规预算；$K = 16/32$ 时分别为 5.61%/5.77%。校准完整选择策略在所有候选数上保持预算内，且 coverage 随可用候选增加而上升。该结果隔离了 arg max 的选择偏差，而不是把真实系统性能与合成样本混为一谈。

为检查有限策略族同时界的实现,我们重复生成 300 个独立校准样本。Hoeffding 界在 299/300 次重复中同时覆盖所有策略–约束项，经验 Bernstein 界为 300/300；二者均高于 90% 名义水平。这一实验验证的是定理的有限策略族实现并隔离候选搜索的影响，但不替代真实物理证据。

表 5在互斥的 666 个校准组和 400 个评测组上比较风险控制方法。相同名义 $\alpha = 0.05$ 下，逐候选边际校准只保留 3.75% coverage，其零事件样本仍不足以证明 5% 条件风险；Bonferroni 全部拒绝。本文选择后分数保留 25% coverage、78% 正收益动作，并把任一约束风险上界控制在 3.83%。未加外层校准的 selector 和 CRC 式方法行动更多，但执行条件风险上界超过 5%；CRC 控制的是不同目标 $\mathbb{E}[AV]$，而不是 $\Pr(V = 1 \mid A = 1)$。完整运行点网格上，本文方法在相近可证风险下也优于逐候选校准，因此收益并非来自无差别增加拒绝。

表 6: 完整选择策略证书与独立的可靠性过滤实机决策执行证据。二者是分别固定的运行点；LCB/UCB 均为精确单侧 90% 界。

统计量	完整选择策略证书	Learned-live 证据
实验组	400	600
动作 coverage	100/400=25.00%（LCB 22.20%）	147/600=24.50%（LCB 22.23%）
任一约束风险	1/100=1.00%（UCB 3.83%）	2/147=1.36%（UCB 3.58%）
严重退化风险	0/100（UCB 2.28%）	0/147（UCB 1.55%）

表 7: 五类真实工作负载上的实机决策 Agent 留出评测。拒绝按 Linux 零增益计。效用区间重采样完整 20-scenario round；coverage 和条件风险精确界使用随机化配对实验组。

指标	结果
有效逻辑 trial	600/600
完整 round blocks	30
非 Linux 动作覆盖率（单侧下界）	147/600=24.50%（22.23%）
所选动作正向收益率	114/147=77.55%
非正收益动作效用（均值 / 最差）	-0.02142 / -0.09608
全体平均有界效用（95% CI）	0.01398 [0.01094, 0.01694]
任一约束违规（单侧上界）	2/147=1.36%（3.58%）
严重调度退化（单侧上界）	0/147（1.55%）
实际使用的非 Linux ActionPlan	7

表 8: 按工作负载分解的固定实机决策结果。效用将拒绝按 Linux 计入；positive 与 events 只在实际执行动作内统计。

Workload	Trials	Acted	Positive	效用	Events
Futex	120	0	0	0.00000	0
MariaDB	120	54	54	0.03636	0
RocksDB	120	31	17	0.00039	2
schbench	120	17	7	-0.00038	0
TinyLlama	120	45	36	0.03355	0

7.4 实机决策 Agent 留出评测

固定策略在 600 个不预知执行结果的 trial 中实际执行 147 次，coverage 为 24.50%，精确单侧 90% 下界为 22.23%。114/147 个动作获得正的 Linux 相对效用。将 453 次拒绝按 Linux 零增益计入后，全部 trial 的平均有界效用为 0.01398，round-block 自助重采样 95% 置信区间为 [0.01094,0.01694]。这些数字从原始配对指标比值按式 5重建, 与训练和排序使用同一有界目标。正下界表明收益不是从事后挑选的个别动作推断而来；每次选择都发生在当前 outcome 产生之前，并与同 trial 的 Linux anchor 形成随机顺序配对。

表 8在不合并工作负载的情况下报告同一固定策略。拒绝按 Linux 零效用计入；正收益与约束事件只在实际执行动作内统计，因此该表同时显示 Agent 的行动分布与总体策略效用。

表 9从目标维度给出互补视图，显示结构化目标会同时改变行动频率和物理性能权衡。无 P99 目标只用于该指标未定义的 futex；固定选择策略因效用下界非正而在这些 trial 上全部拒绝。

七种非 Linux 动作均在真实 trial 中执行：六个 scx_adapt 资源方案分别被选择 2、13、30、1、17 和 2 次，scx_simple 被选择 82 次，因此策略没有退化为单一优秀动作的固定路由。33 个非正收益动作的 bounded-symmetric 效用均值为 -0.02142、中位数为 -0.00857、最差为 -0.09608。两个约束事件均为 SLO 违规。以每个

表 9: 按结构化目标分解的固定实机决策结果。效用将拒绝按 Linux 计入；positive 与 events 只在实际执行动作内统计。

目标	Trials	Acted	Positive	效用	Events
平衡	92	24	22	0.00834	0
公平性预算	94	3	1	−0.00007	0
尾延迟优先	99	43	32	0.02928	1
SLO 与后台	85	36	28	0.01347	1
吞吐与后台（无 P99）	120	0	0	0.00000	0
吞吐优先	110	41	31	0.03260	0

随机化 candidate–Linux 配对作为一个实验组，任一约束与严重退化风险的精确单侧 90% 上界分别为 3.58% 和 1.55%，满足预先确定方案的的 5% 与 2% 运行预算。30 个 workload-balanced round 只用于效用自助重采样，不替代风险的实验组统计单位。Guard 不参与候选效用排序，故这些结果衡量的是固定 Agent 的观测到动作预测、目标组合、选择性拒绝和执行路径，而不是运行时事后筛掉不利 outcome。独立的 400 组证书在打开评测角色前绑定候选生成、排序、margin filter、拒绝和 Linux 回退全过程，并与其校准组零重叠。该证书证明 5% 任一约束预算与非零 coverage；更多实际运行样本则提供 2% 严重退化指标所需的统计精度。

7.5 动作条件不确定性

在独立 P99 诊断中，MariaDB、RocksDB、schbench 和 TinyLlama 的 90% 区间经验 coverage 分别为 96.33%、91.78%、95.95% 和 93.24%，每类均包含至少 30 个独立 candidate–baseline pairs，且没有工作负载低于 85% 诊断下界。MariaDB 和 schbench 的区间略偏保守。逐指标区间用于描述尾延迟数值不确定性；选择后风险判断仍使用最终策略上的真实约束事件界，而不是把边际 coverage 直接解释为通用安全保证。

7.6 用户态决策路径开销

效果预测在 CPU 上运行，目标 composer 只执行确定性向量运算。在独立的缓存式策略路径微基准中，单次决策 P95 为 4.01 ms，目标 composer P95 为 0.30 ms。该微基准只测量用户态预测与目标组合路径，不包含 eBPF 观测或动作应用/回退延迟，也不用于估计表 7中的策略效用；其开销位于秒级控制周期之外，不进入内核快速调度路径。由于高频决策不依赖外部 LLM API，网络超时不会改变当前调度状态；自然语言解析失败时，Agent 保持上一个经验证的结构化目标或回退 Linux。

8 相关工作

内核控制智能体与学习式调度选择。 Mixture-of-Schedulers 将负载识别与设备相关的调度器映射解耦，运行时以学习式分类器和时间投票选择专家调度器 [19]。pMVX 并行运行隔离 OS 策略变体，并把 eBPF 负载特征到有效策略的关系压缩为轻量策略记忆 [14]。SchedCP 则以 LLM Agent 分析工作负载、生成 sched_ext 策略并执行静态与动态验证 [21]。KernAdapt 不生成调度代码，也不把完整 scheduler identity 作为唯一动作；它学习可组合内核资源动作的向量效果，根据新目标重组选择，并显式处理优化选择后的约束风险。

组合动作学习。 Branching DQN 通过共享状态表示和动作维度分支缓解高维离散动作空间的组合爆炸 [17]。因子化 Q 函数、参数化动作和组合 bandit 进一步利用动作结构共享样本。动作分解本身并非本文的新颖点；KernAdapt 关注的是昂贵物理系统干预中的多指标相对效果、机制允许交互、完整配置组合留出和无支持组合拒绝。效果先于目标的建模还允许同一动作结果服务训练外目标权重。

离线策略学习与干预反馈。 IPS、doubly robust 估计和 counterfactual risk minimization 利用日志 propensity 学习目标策略 [3, 15]，CQL 等保守离线强化学习方法抑制分布外动作价值高估 [8]。KernAdapt 的数据由 Linux 基线随机不完全块主动产生，propensity 已知，结果是带物理相关性的同一实验组内多个动作的结果。本文因此以实验组聚合动作执行，再利用支持结构处理未见完整动作。

共形预测与策略风险。 共形预测为交换数据上的预测集合提供有限样本边际覆盖 [1]；Conformal Risk Control 推广到单调损失的期望风险 [2]；COPP 研究上下文 bandit 的离策略结果区间 [16]；CAP 学习输入相关的拒绝策略 [18]；Conformal Policy Control 以安全参考策略调节新策略的行为偏离 [11]。KernAdapt 的统计对象是上下文相关合法集合上的候选生成–arg max–接受/拒绝全过程，并同时报告联合风险、执行条件风险和动作覆盖率。本文的有限族同时界建立在随机实验组和已知动作包含概率上，而非声称一般共形方法都无法处理选择。

工具智能体、约束与本地模型。 ReAct 交错推理、工具调用和反馈 [20]，ToolEmu 揭示高权限工具智能体的长尾风险 [13]，SFT 和偏好优化则利用示范或偏好塑造语言模型行为 [10, 12]。KernAdapt 中的结构化约束和本地语义模块继承了可验证工具接口的思想，但动作价值不由语言模型自评；真实随机系统结果始终是效用和约束标签的来源。

可编程内核机制。 Caladan、ghOSt 和 sched_ext 提供灵活调度与资源控制机制 [4, 6, 9]，eBPF 提供低开销可编程观测 [5]。这些系统回答如何实现快速、灵活的机制；KernAdapt 把它们作为智能体的状态、动作和反馈环境，研究如何在多个合法机制组合之间学习效果与决策边界。

9 讨论

组合泛化的适用域。 两个学习模型只对因子水平和关键因子对具有训练支持、且状态位于预先确定的训练分布范围内的未见完整配置组合进行结构化外推。完全无支持的机制值或状态会触发拒绝。该设计把“未见完整组合”与“没有任何相关证据”区分开来，避免用宽泛的零样本措辞掩盖外推假设。当前 exact-held 证据来自 E1，并且 Critic 使用工作负载与配置指示量，因此不蕴含对未见任务或机器的迁移能力。

风险保证的对象。 定理 1 控制的是随机不完全块分布下、固定策略族中的统计约束事件，不是任意内核错误的形式安全证明。保证依赖实验组独立性、正确 propensity、overlap 和校准分布稳定性。Guard、Canary 与 Rollback 提供互补的能力安全和影响边界，但不改变这些统计假设。

完整选择策略证书在打开互斥评测组前绑定候选生成、排序、margin filter、拒绝和 Linux 回退。正文中的风险界以随机化配对实验组为单位；完整 round 只用于效用区间，以保留采集时的时间分块结构。

顺序干预。 Linux 基线与动作顺序随机化降低公共漂移和估计方差，却不能复现同一时刻全部动作的潜在结果。温度、DVFS、缓存、前一个动作和残留状态清理时长应作为组级协变量或敏感性分析对象。本文因此采用“配对干预效果”，而不声称获得无条件的个体反事实。

语言模型的角色。 KernAdapt 的高频策略由本地结构化模型执行。小型语言模型可以通过 SFT 或教师蒸馏增强自然语言目标解析、约束检索与解释，但它不是数值效果真值，也不是动作执行器。这样的分层允许语言能力随计算资源增强，同时保持无网络环境下的基本控制能力。

反馈与在线适应。 KernAdapt 运行时保留反馈、记忆和重决策接口；为防止测试期更新混淆效用归因，本文留出实验固定学习策略。因此，当前结果验证每次方案选择及其执行效果，不把收益归因于在线更新或长期记忆机制本身。

10 结论

本文将本地内核资源控制表述为 AI 智能体的面向不同目标的组合决策问题。KernAdapt 从以 Linux 为基线的分组随机干预中分别学习供约束检查使用的多指标动作效果和与目标相关的配对效用，并通过结构化动作因子组合互补专家，再对完整候选搜索与选择性拒绝策略构造组级多约束风险界。原有 KernAdapt Agent 的观察、规划、Critic、记忆与重决策接口得到保留；留出评测固定学习策略，使收益不依赖测试期更新。eBPF、sched_ext、cgroup 和可信执行状态机则提供真实反馈与独立权限边界。一次性 560 组 E1 完整配置组合留出评测显示，因子化组合模型相对强因子化结果预测 GBDT 和 pMVX 式检索分别将 regret 降低 30.95% 和 35.36%，两项实验组配对置信下界均为正。模型固定后的 Agent 在 600 次实际运行中以 24.50% coverage 取得正的总体有界效用，其完整 round 自助重采样区间严格大于零；实际执行实验组上的任一约束与严重退化单侧上界分别为 3.58% 和 1.55%。该方法把可编程内核机制中缺失的部分从“再设计一个调度器”转向“学习何时以及按何种目标使用一组结构化系统工具”。

参考文献

[1] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations*, 2024.

[3] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *International Conference on Machine Learning*, 2011.

[4] Joshua Fried, Zhenyuan Ruan, Adam Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *USENIX Symposium on Operating Systems Design and Implementation*, 2020.

[5] Brendan Gregg. BPF performance tools: Linux system and application observability. In *USENIX LISA*, 2019.

[6] Jack Tigar Humphries et al. ghOSt: Fast and flexible user-space delegation of linux scheduling. In *ACM Symposium on Operating Systems Principles*, 2021.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

[8] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[9] Linux Kernel Documentation. Extensible scheduler class. `https://docs.kernel.org/scheduler/sched-ext.html`, 2024.

[10] Long Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[11] Drew Prinster, Clara Fannjiang, Ji Won Park, Kyunghyun Cho, Anqi Liu, Suchi Saria, and Samuel Stanton. Conformal policy control. *arXiv preprint arXiv:2603.02196*, 2026.

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[13] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

[14] Sujot Singh, Eddie Federmeyer, Kenan Alghythee, and Xiaoguang Wang. pMVX: Policy-level multi-version execution for agentic OS kernel self-tuning. In *Workshop on Operating Systems for AI Agents*, 2026.

[15] Adith Swaminathan and Thorsten Joachims. Counterfactual risk minimization: Learning from logged bandit feedback. In *International Conference on Machine Learning*, 2015.

[16] Muhammad Faaiz Taufiq, Jean-Francois Ton, Rob Cornish, Yee Whye Teh, and Arnaud Doucet. Conformal off-policy prediction in contextual bandits. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[17] Arash Tavakoli, Fabio Pardo, and Petar Kormushev. Action branching architectures for deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[18] Sina Tayebati, Divake Kumar, Nastaran Darabi, Dinithi Jayasuriya, Theja Tulabandhula, Ranganath Krishnan, and Amit Ranjan Trivedi. CAP: Conformalized abstention policies for context-adaptive risk management for LLMs and VLMs. In *Asian Conference on Machine Learning*, volume 304 of *Proceedings of Machine Learning Research*, pages 926–941, 2025.

[19] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, and Mingli Song. Mixture-of-schedulers: An adaptive scheduling agent as a learned router for expert policies. *arXiv preprint arXiv:2511.11628*, 2025.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[21] Yusheng Zheng, Yanpeng Hu, Wei Zhang, and Andi Quinn. Towards agentic OS: An LLM agent framework for linux schedulers. *arXiv preprint arXiv:2509.01245*, 2025.

A 完整动作目录

表 10是实机决策使用的八选项目录；Linux 与 scx_simple 是参考动作，其余六项是登记的 scx_adapt ActionPlan。表 11是另一套独立的八个完整组合；Linux 在每个随机块中作为配对锚点，不属于八个被留出的元组。

表 10: 实机决策八选项目录。

Action ID	调度后端	队列/placement	slice/preemption	idle/affinity	quota/weight
baseline.linux_default	Linux	kernel/preserve	kernel/kernel	kernel/unchanged	unlimited/default
baseline.scx_simple	scx_simple	kernel/preserve	kernel/kernel	kernel/unchanged	unlimited/default
adapt.latency	scx_adapt	global/load-aware	short/wakeup	prefer-idle/spread	high/primary
adapt.throughput	scx_adapt	per-CPU/preserve	long/off	preserve/unchanged	unlimited/balanced
adapt.isolation	scx_adapt	per-CPU/compact	medium/wakeup	prefer-idle/isolated	high/primary
adapt.fairness	scx_adapt	per-cgroup/spread	medium/off	prefer-idle/spread	unlimited/balanced
adapt.background_progress	scx_adapt	per-cgroup/load-aware	long/off	preserve/spread	unlimited/background
adapt.efficiency	scx_adapt	global/compact	long/off	preserve/compact	medium/primary

表 11: 独立的八个 exact-held 完整组合；Linux 不是留出元组。

Action ID	调度后端	队列/placement	slice/preemption	idle/affinity	quota/weight
composition.c01	scx_adapt	global/load-aware	long/off	prefer-idle/spread	high/primary
composition.c02	scx_adapt	per-CPU/preserve	medium/wakeup	preserve/unchanged	unlimited/balanced
composition.c03	scx_adapt	per-CPU/compact	medium/off	preserve/isolated	unlimited/background
composition.c04	scx_adapt	per-cgroup/spread	long/off	prefer-idle/spread	medium/primary
composition.c05	scx_adapt	per-cgroup/load-aware	long/off	preserve/spread	high/primary
composition.c06	scx_adapt	global/compact	short/wakeup	prefer-idle/compact	unlimited/balanced
composition.c07	scx_adapt	per-CPU/preserve	medium/off	prefer-idle/unchanged	medium/primary
composition.c08	scx_adapt	global/compact	medium/off	prefer-idle/compact	unlimited/background